

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Основы алгоритмизации и программирования

К ЗАЩИТЕ ДОПУСТИТЬ
_____ Ковальчук А. М.

Отчет
по учебной практике (ознакомительной)
по теме

СПРАВКА ЖЕЛЕЗНОДОРОЖНОГО ВОКЗАЛА

БГУИР КР 6-05-0611-05 117 ПЗ

Студент

Мистюкович Д.В. (гр. 458301)

Руководитель

Ковальчук А. М.

МИНСК 2025

СОДЕРЖАНИЕ

УСЛОВИЕ ЗАДАНИЯ.....	3
ВВЕДЕНИЕ.....	4
1 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ.....	5
1.1 Входные и выходные данные.....	5
2 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ.....	6
2.1 Разработка схем алгоритмов.....	6
2.2 Разработка алгоритмов.....	6
3 РЕЗУЛЬТАТ РАБОТЫ.....	8
ЗАКЛЮЧЕНИЕ.....	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	11
ПРИЛОЖЕНИЕ А.....	12
ПРИЛОЖЕНИЕ Б.....	13
ПРИЛОЖЕНИЕ В.....	14
ПРИЛОЖЕНИЕ Г.....	15
ПРИЛОЖЕНИЕ Д.....	17
ПРИЛОЖЕНИЕ Е.....	19
ПРИЛОЖЕНИЕ Ж.....	27
ПРИЛОЖЕНИЕ З.....	34

УСЛОВИЕ ЗАДАНИЯ

Программа по запросу выдает справку о маршруте, т.е. время рейса, расстояние между городами. Если нет прямого пути между городами, то указываются станции пересадок. Информация должна храниться в файлах.

ВВЕДЕНИЕ

Программа помогает пользователю быстро находить маршруты между городами. Она показывает время в пути, расстояние и пересадки, если прямого рейса нет. Данные хранятся в файлах для удобства и быстрого доступа.

Программа реализована на языке C, что обеспечивает высокую скорость работы и эффективное использование системных ресурсов. Для удобства пользователей реализован простой и интуитивно понятный интерфейс, позволяющий быстро получать нужную информацию.

В случае отсутствия прямого сообщения между городами, система автоматически строит оптимальный маршрут с пересадками, рассчитывая общее время в пути и суммарное расстояние.

1 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

1.1 Изначальные данные

Изначальные данные:

```
struct ring_node      // Элемент кольца (рейс)
{
    char start[50];    // Начальный город
    char end[50];      // Конечный город
    int distance;      // Расстояние между городами
    int direction;     // Направление рейса
    int time;          // Время рейса в минутах
    struct ring_node* next;
};

FILE* file;           //файл для хранения и записи рейсов
```

2 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

2.1 Разработка схем алгоритмов

Схема алгоритма программы main.cpp представлена в приложении А. Функция `add_flight(ring_node* head)` позволяет добавлять новые рейсы, схема алгоритма представлена в приложении Б. Функция `load_ring_from_file(const char* filename)` позволяет загружать рейсы из бинарного файла, схема алгоритма представлена в приложении В.

2.2 Разработка алгоритмов

Функция `char* find_shortest_path(ring_node* head)` позволяет пользователю найти кратчайший путь между городами.

Передаваемые параметры:

`ring_node* head` – указатель на голову кольца хранящее все рейсы.

Шаг 1. Начало.

Шаг 2. Объявить переменные:

`char** cities` – список уникальных городов

`int city_count` – количество городов

`int** distance_graph` – граф для хранения расстояний между городами

`int** time_graph` – граф для хранения времени рейсов между городами

`int start, end` – начальный и конечный город

`int* dist` – массив для хранения минимального расстояния от
изначального города до всех остальных

`int* time` – массив для хранения минимального времени в пути от
начального города до всех остальных городов

`int* prev` – массив для хранения информации о предыдущих вершинах

`char* result` - результат

Шаг 3. Считаем колво городов и записываем в `city_count` и записываем города в `cities`, при помощи функции:

`city_count = read_cities_from_ring(head, &cities)`

Шаг 4. Создаем графы, при помощи функций:

`distance_graph = create_graph(city_count);`

`time_graph = create_graph(city_count);`

Шаг 5. Заполняем графы, при помощи функции:

`build_flight_graph(head, cities, city_count, distance_graph, time_graph);`

Шаг 6. Вводим начальный и конечный город и записываем в `start, end`

Шаг 7. Выделяем память под массивы `dist, time, prev` размером `city_count`

Шаг 8. Считаем кратчайший путь, при помощи функции:

`dijkstra_algorithm(distance_graph, time_graph, city_count, start, dist, time, prev);`

Шаг 9. Записываем результат при помощи функции:

```
char* result = build_path_result(cities, dist, time, prev, city_count, end);
```

Шаг 10. Очищаем всю выделенную память, при помощи функции
`free_memory(cities, distance_graph, time_graph, dist, time, prev, city_count);`

Шаг 11. Вернуть результат (result).

Шаг 12. Конец.

Функция `save_ring_to_file(const char* filename, ring_node* head)`
позволяет загружать данные в бинарный файл.

Передаваемые параметры:

`const char* filename` – название бинарного файла

`ring_node* head` – указатель на голову кольца

Шаг 1. Начало.

Шаг 2. Объявить переменные:

`FILE* file_bin` – бинарный файл

`ring_node* current = head` – текущий элемент кольца

Шаг 3. Открыть бинарный файл с именем filename

Шаг 4. Если head == NULL то 4 и 5 иначе 6

Шаг 5. Закрыть файл

Шаг 6. Конец

Шаг 7. Цикл do-while пока current != head

Шаг 8. Записываем информацию из элемента кольца в бинарный файл

Шаг 9. Переходим к следующему элементу кольца

`current = current->next;`

Шаг 10. Конец цикла

Шаг 11. Закрыть файл

Шаг 12. Конец.

3 РЕЗУЛЬТАТ РАБОТЫ

```
Меню:  
1. Добавить рейс  
2. Показать рейсы  
3. Удалить рейс по номеру  
4. Найти кратчайший путь  
5. Сохранить изменения в файл  
0. Выход  
Ваш выбор: █
```

Рисунок 3.1 – меню программы

```
Введите начальный город: Минск  
Введите конечный город: Гродно  
Введите расстояние (в км): 253  
Введите направление (0 – оба, 1 – туда, 2 – обратно): 0  
Введите время рейса (в минутах): 125  
Рейс добавлен.  
Нажмите любую кнопку для продолжения... █
```

Рисунок 3.2 – ввод рейса

```
Выберите способ отображения:  
1 - Все рейсы  
2 - Рейсы из указанного города  
3 - Рейсы в указанный город  
Ваш выбор: █
```

Рисунок 3.3 – выбор отображения рейсов

Список всех рейсов:				
№	:Начало -> конец	расстояние (км)	время в пути (минут)	Направление
1	:Кричев -> Слоним	90	96	Обратно
2	:Брест -> Минск	86	101	Туда
3	:Гомель -> Витебск	59	56	Туда
4	:Брест -> Гомель	336	305	Туда
5	:Гомель -> Полоцк	97	119	Оба направления
6	:Могилев -> Минск	459	399	Оба направления
7	:Любань -> Борисов	279	304	Обратно
8	:Мозырь -> Калинковичи	81	78	Обратно
9	:Мозырь -> Витебск	121	117	Обратно
10	:Молодечно -> Минск	143	153	Оба направления

Рисунок 3.3.1 – вывод всех рейсов

Введите город: Минск

Результаты поиска:

№	:Начало -> конец	расстояние (км)	время в пути (минут)	Направление
1	:Минск -> Жодино	179	239	Оба направления
2	:Минск -> Могилев	328	285	Оба направления
3	:Минск -> Пинск	285	295	Обратно
4	:Минск -> Витебск	297	356	Обратно
5	:Минск -> Полоцк	407	376	Туда
6	:Минск -> Орша	407	428	Оба направления
7	:Минск -> Солигорск	100	133	Обратно
8	:Минск -> Могилев	218	190	Туда
9	:Минск -> Гродно	55	48	Туда
10	:Минск -> Речица	241	258	Обратно

Рисунок 3.3.2 – вывод рейсов из указанного города

Введите город: Минск

Результаты поиска:

№	:Начало -> конец	расстояние (км)	время в пути (минут)	Направление
1	:Брест -> Минск	86	101	Туда
2	:Могилев -> Минск	459	399	Оба направления
3	:Молодечно -> Минск	143	153	Оба направления
4	:Витебск -> Минск	294	289	Оба направления
5	:Гомель -> Минск	484	519	Туда
6	:Могилев -> Минск	379	334	Туда
7	:Витебск -> Минск	272	281	Туда
8	:Полоцк -> Минск	338	356	Оба направления
9	:Брест -> Минск	487	430	Оба направления
10	:Гомель -> Минск	116	110	Оба направления

Рисунок 3.3.3 – вывод рейсов в указанный города

Введите название начального города: Калинковичи
Введите название конечного города: Рогачёв
Маршрут: Калинковичи -> Мозырь -> Осиповичи -> Борисов -> Рогачёв
Расстояние: 474 км
Время в пути : 494 мин. (8, 2 ч.)
Нажмите любую кнопку для продолжения...

Рисунок 3.4 – нахождение кратчайшего пути между городами

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была успешно разработана программа для поиска маршрутов между городами. Приложение предоставляет пользователям точную информацию о времени в пути, расстоянии между населёнными пунктами и вариантах пересадок при отсутствии прямого рейса.

Программа имеет удобный и интуитивно понятный интерфейс, что значительно упрощает её использование. Чёткая структура меню, наглядное отображение маршрутов и простая система управления позволяют пользователям быстро освоить основные функции приложения.

Особое внимание при разработке было уделено оптимизации производительности. Эффективная работа с файлами данных и продуманные алгоритмы поиска обеспечивают высокую скорость работы приложения даже при обработке больших объёмов информации.

В результате выполнения проекта были не только закреплены навыки программирования на языке C, но и получен ценный опыт создания реального приложения для работы с транспортными маршрутами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы алгоритмизации и программирования : лаборатор. практикум для студентов специальности 1-40 02 01 «Вычисл. машины, системы и сети» всех форм обучения. В 2 ч. Ч. 2 / сост. Ю. А. Луцик [и др.]. –Минск : БГУИР, 2010. – 36 с. : ил.
2. StackOverflow [Электронный ресурс] – Вопросы – режим доступа: <https://ru.stackoverflow.com/questions>.

ПРИЛОЖЕНИЕ А
(обязательное)

Схема алгоритма программы main.cpp

ПРИЛОЖЕНИЕ Б
(обязательное)

Схема алгоритма функции load_ring_from_file()

ПРИЛОЖЕНИЕ В
(обязательное)

Схема алгоритма функции `add_flight(ring_node* head)`

ПРИЛОЖЕНИЕ Г

(обязательное)

Код файла main.cpp

```
#include "header.h"
int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    setlocale(LC_ALL, "");
    ring_node* head = load_ring_from_file("flights.bin");
    while (1)
    {
        printf("Меню:\n1. Добавить рейс\n2. Показать рейсы\n3. Удалить рейс по номеру\n4. Найти кратчайший путь\n5. Сохранить изменения в файл\n0. Выход\nВаш выбор: ");
        int choice = input_int_number();
        char* res;
        switch (choice)
        {
            case 1:
                system("cls");
                head = add_flight(head);
                break;

            case 2:
                system("cls");
                printf("Выберите способ отображения:\n1 - Все рейсы\n2 - Рейсы из указанного города\n3 - Рейсы в указанный город\nВаш выбор: ");
                choice = input_int_number();
                if (choice == 1)
                    print_flights(head);
                else if (choice == 2 || choice == 3)
                    find_flights_by_city(head, choice);
                else
                    printf("Неверный выбор.\n");
                break;

            case 3:
                system("cls");
                head = delete_flight_by_number(head);
                break;
```

```

case 4:
    system("cls");
    res = find_shortest_path(head);
    printf("%s", res);
    free(res);
    break;

case 5:
    system("cls");
    save_ring_to_file("flights.bin", head);
    break;

case 0:
    if (exit_())
    {
        save_ring_to_file("flights.bin", head);
        free_ring(head);
        return 0;
    }
    else
    {
        system("cls");
        continue;
    }

default:
    printf("Вы ввели неправильные значения, попробуйте снова.");
    break;
}
printf("Нажмите любую кнопку для продолжения...");
getchar();
system("cls");
}
return 0;
}

```


ПРИЛОЖЕНИЕ Д

(обязательное)

Код файла header.h

```
#ifndef HEADER_H
#define HEADER_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <locale.h>
#include <limits.h>
#include <Windows.h>

struct ring_node      // Информация о рейсе
{
    char start[50];    // Начальный город
    char end[50];      // Конечный город
    int distance;       // Расстояние между городами
    int direction;      // Направление 0 - оба направления, 1 - туда, 2 -
    обратно
    int time;           // Время рейса в минутах
    struct ring_node* next;
};

// Проверка на выход из программы
int exit_();
// Ввод числа
int input_int_number();
// Создание элемента кольца
ring_node* create_ring(const char* start, const char* end, int distance, int
direction, int time);
// Добавление элемента кольца
ring_node* add_flight_to_ring(ring_node* head, ring_node* new_node);
// Удаление элемнета кольца по номеру
ring_node* delete_flight_by_number(ring_node* head);
// Очистка памяти из под кольца
void free_ring(ring_node* head);
// Вывод всех рейсов
void print_flights(ring_node* head);
// Поиск рейсов по городу
void find_flights_by_city(ring_node* head, int search_type);
```

```

// Добавление рейса
ring_node* add_flight(ring_node* head);
// Сохранение данных в файл
void save_ring_to_file(const char* filename, ring_node* head);
// Загрузка данных из файла
ring_node* load_ring_from_file(const char* filename);
// Создание списка для городов
char** create_cities_array(int size);
// Создание графов
int** create_graph(int size);
// Запись городов в список
int read_cities_from_ring(ring_node* head, char*** cities_ptr);
// Заполнение графов
void build_flight_graph(ring_node* head, char** cities, int city_count, int**
distance_graph, int** time_graph);
// Основная функция нахождения кратчайшего пути
char* find_shortest_path(ring_node* head);
// Алгоритм Дейкстры
void dijkstra_algorithm(int** graph, int** time_graph, int city_count, int start, int*
dist, int* time, int* prev);
// Создание результата
char* build_path_result(char** cities, int* dist, int* time, int* prev, int city_count,
int end);
// Очистка памяти из под алгоритма Дейкстры
void free_memory(char** cities, int** distance_graph, int** time_graph, int* dist,
int* time, int* prev, int cities_count);
#endif

```

ПРИЛОЖЕНИЕ Е

(обязательное)

Код файла function.cpp

```
#include "header.h"

int exit_()
{
    char prov;
    while (1)
    {
        printf("\nВы уверены что хотите закончить? (Y - да, N - нет):");
        prov = getchar();

        if (toupper(prov) == 'Y')
            return 1;
        else if (toupper(prov) == 'N')
        {
            while (getchar() != '\n');
            system("cls"); // Очистка консоли
            return 0;
        }
        else
        {
            puts("Неправильный ввод. Попробуйте снова.\n");
            while (getchar() != '\n');
        }
    }
}

int input_int_number()
{
    int n;
    if (scanf_s("%d", &n) == 1 && n >= 0)
    {
        while (getchar() != '\n');
        return n;
    }
    else
    {
        puts("Неправильный ввод. Введите положительное число.");
        while (getchar() != '\n');
        input_int_number();
    }
}
```

```

ring_node* create_ring(const char* start, const char* end, int distance, int
direction, int time)
{
    ring_node* node = (ring_node*)malloc(sizeof(ring_node));

    strcpy_s(node->start, sizeof(node->start), start);
    strcpy_s(node->end, sizeof(node->end), end);

    node->distance = distance;
    node->direction = direction;
    node->time = time;
    node->next = NULL;

    return node;
}
ring_node* add_flight_to_ring(ring_node* head, ring_node* new_node)
{
    if (!head)
    {
        new_node->next = new_node;
        return new_node;
    }

    ring_node* tail = head;
    while (tail->next != head)
        tail = tail->next;

    tail->next = new_node;
    new_node->next = head;

    return head;
}
ring_node* delete_flight_by_number(ring_node* head)
{
    if (head == NULL)
    {
        printf("Список рейсов пуст.\n");
        return NULL;
    }

    print_flights(head);
}

```

```

printf("Введите номер рейса для удаления: ");
int flightNumber = input_int_number();

if (flightNumber < 1)
{
    printf("Неверный номер рейса.\n");
    return head;
}

struct ring_node* current = head;
struct ring_node* prev = NULL;
int count = 1;

while (count < flightNumber)
{
    prev = current;
    current = current->next;
    count++;

    if (current == head && count <= flightNumber)
    {
        printf("Рейс с номером %d не найден.\n", flightNumber);
        return head;
    }
}

if (current->next == current)
{
    free(current);
    return NULL;
}
else
{
    if (current == head)
    {
        struct ring_node* temp = head;
        while (temp->next != head)
            temp = temp->next;
        temp->next = head->next;
        head = head->next;
    }
    else
        prev->next = current->next;
    free(current);
}

```

```

    }

    printf("Рейс %d удален.\n", flightNumber);
    return head;
}
void free_ring(ring_node* head)
{
    if (head == NULL) return;

    struct ring_node* current = head;
    struct ring_node* next_node;

    do
    {
        next_node = current->next;
        free(current);
        current = next_node;
    } while (current != head);
}

void print_flights(ring_node* head)
{
    if (head == NULL)
    {
        printf("Список рейсов пуст.\n");
        return;
    }

    printf("\nСписок всех рейсов:\n");
    printf("№ :Начало -> конец%s| расстояние (км) | время в пути (минут) |  

Направление\n", 15, " ", 5, " ");

    struct ring_node* current = head;
    int count = 0;

    do
    {
        count++;
        printf("%-3d:%s -> %s| %-15d | %-20d | ",
            count, current->start, (-26 + (int)strlen(current->start)), current->end,
            current->distance, current->time);

        switch (current->direction)
        {

```

```

    case 0: printf("Оба направления\n"); break;
    case 1: printf("Туда\n"); break;
    case 2: printf("Обратно\n"); break;
    default: printf("Неизвестно\n");
}

    current = current->next;
} while (current != head);

printf("\nВсего найдено рейсов: %d\n", count);
}
void find_flights_by_city(ring_node* head, int search_type)
{
    if (head == NULL)
    {
        printf("Список рейсов пуст.\n");
        return;
    }

    char city[50];
    printf("Введите город: ");
    gets_s(city, 50);

    printf("\nРезультаты поиска:\n");
    printf("№ :Начало -> конец%s| расстояние (км) | время в пути (минут) |
Направление\n", 15, " ", 5, " ");

    struct ring_node* current = head;
    int count = 0;

    do
    {
        int match = 0;
        if (search_type == 2)
            match = (strcmp(current->start, city) == 0);
        else if (search_type == 3)
            match = (strcmp(current->end, city) == 0);

        if (match)
        {
            count++;
            printf("%-3d:%s -> %s| %-15d | %-20d | ",
                count, current->start, (-26 + (int)strlen(current->start)), current->end,
                current->distance, current->time);

```

```

        switch (current->direction)
        {
            case 0: printf("Оба направления\n"); break;
            case 1: printf("Туда\n"); break;
            case 2: printf("Обратно\n"); break;
            default: printf("Неизвестно\n");
        }
    }

    current = current->next;
} while (current != head);

if (count == 0)
    printf("\nРейсов не найдено.\n");
else
    printf("\nВсего найдено рейсов: %d\n", count);
}

ring_node* add_flight(ring_node* head)
{
    char start[50], end[50];
    int distance, direction, time;

    printf("Введите начальный город: ");
    gets_s(start, sizeof(start));

    printf("Введите конечный город: ");
    gets_s(end, sizeof(end));

    printf("Введите расстояние (в км): ");
    distance = input_int_number();

    do
    {
        printf("Введите направление (0 — оба, 1 — туда, 2 — обратно): ");
        direction = input_int_number();
        if (direction < 0 || direction > 2)
            printf("Неверное значение. Повторите ввод.\n");
    } while (direction < 0 || direction > 2);

    printf("Введите время рейса (в минутах): ");
    time = input_int_number();

    ring_node* new_node = create_ring(start, end, distance, direction, time);

```



```

    head = add_flight_to_ring(head, new_node);

    printf("Рейс добавлен.\n");
    return head;
}

void save_ring_to_file(const char* filename, ring_node* head)
{
    FILE* file;
    fopen_s(&file, filename, "wb");

    if (head == NULL)
    {
        int zero = 0;
        fwrite(&zero, sizeof(int), 1, file);
        fclose(file);
        return;
    }

    int count = 0;
    ring_node* current = head;
    do
    {
        count++;
        current = current->next;
    } while (current != head);

    fwrite(&count, sizeof(int), 1, file);

    current = head;
    do
    {
        fwrite(current->start, sizeof(current->start), 1, file);
        fwrite(current->end, sizeof(current->end), 1, file);
        fwrite(&current->distance, sizeof(int), 1, file);
        fwrite(&current->direction, sizeof(int), 1, file);
        fwrite(&current->time, sizeof(int), 1, file);

        current = current->next;
    } while (current != head);

    fclose(file);
}

```

```

ring_node* load_ring_from_file(const char* filename)
{
    FILE* file;
    fopen_s(&file, filename, "rb");

    int count;
    fread(&count, sizeof(int), 1, file);

    ring_node* head = NULL;

    for (int i = 0; i < count; i++)
    {
        char start[50], end[50];
        int distance, direction, time;

        fread(start, sizeof(start), 1, file);
        fread(end, sizeof(end), 1, file);
        fread(&distance, sizeof(int), 1, file);
        fread(&direction, sizeof(int), 1, file);
        fread(&time, sizeof(int), 1, file);

        ring_node* new_node = create_ring(start, end, distance, direction, time);
        head = add_flight_to_ring(head, new_node);
    }

    fclose(file);
    return head;
}

```

ПРИЛОЖЕНИЕ Ж

(обязательное)

Код файла dijkstra_algorithm.cpp

```
#include "header.h"

char** create_cities_array(int size)
{
    char** cities = (char**)malloc(size * sizeof(char*));
    for (int i = 0; i < size; i++)
        *(cities + i) = (char*)malloc(50 * sizeof(char));
    return cities;
}

int** create_graph(int size)
{
    int** graph = (int**)malloc(size * sizeof(int*));

    for (int i = 0; i < size; i++)
    {
        *(graph + i) = (int*)malloc(size * sizeof(int));
        for (int j = 0; j < size; j++)
            (*(graph + i) + j) = INT_MAX;
    }
    return graph;
}

int read_cities_from_ring(ring_node* head, char*** cities_ptr)
{
    int city_count = 0;
    int capacity = 10;
    char** cities = (char**)malloc(capacity * sizeof(char*));

    ring_node* current = head;
    do
    {
        int i;
        for (i = 0; i < city_count; i++)
            if (strcmp(*(cities + i), current->start) == 0)
                break;

        if (i == city_count)
        { // Город не найден
            if (city_count >= capacity)
            {
```

```

        capacity += 1;
        char** new_cities = (char**)realloc(cities, capacity * sizeof(char*));
        cities = new_cities;
    }

    *(cities + city_count) = (char*)malloc(50 * sizeof(char));
    strcpy_s(*(cities + city_count), 50, current->start);
    city_count++;
}

for (i = 0; i < city_count; i++)
    if (strcmp(*(cities + i), current->end) == 0)
        break;

if (i == city_count)
{
    if (city_count >= capacity)
    {
        capacity += 1;
        char** new_cities = (char**)realloc(cities, capacity * sizeof(char*));
        cities = new_cities;
    }

    *(cities + city_count) = (char*)malloc(50 * sizeof(char));
    strcpy_s(*(cities + city_count), 50, current->end);
    city_count++;
}

    current = current->next;
} while (current != head);

*cities_ptr = cities;
return city_count;
}

void build_flight_graph(ring_node* head, char** cities, int city_count, int**
distance_graph, int** time_graph)
{
    ring_node* current = head;
    do
    {
        int start_idx = -1, end_idx = -1;
        for (int i = 0; i < city_count; i++)
        {
            if (strcmp(*(cities + i), current->start) == 0) start_idx = i;

```

```

        if (strcmp(*(cities + i), current->end) == 0) end_idx = i;
    }

    if (start_idx != -1 && end_idx != -1)
    {
        if (current->direction == 0 || current->direction == 1)
        {
            (*(distance_graph + start_idx) + end_idx) = current->distance;
            (*(time_graph + start_idx) + end_idx) = current->time;
        }
        if (current->direction == 0 || current->direction == 2)
        {
            (*(distance_graph + end_idx) + start_idx) = current->distance;
            (*(time_graph + end_idx) + start_idx) = current->time;
        }
    }

    current = current->next;
} while (current != head);
}

char* find_shortest_path(ring_node* head)
{
    if (!head)
    {
        printf("Список рейсов пуст.\n");
        return NULL;
    }

    char** cities = NULL;
    int city_count = read_cities_from_ring(head, &cities);

    int** distance_graph = NULL;
    int** time_graph = NULL;

    distance_graph = create_graph(city_count);
    time_graph = create_graph(city_count);

    build_flight_graph(head, cities, city_count, distance_graph, time_graph);

    printf("Доступные города:\n");
    for (int i = 0; i < city_count; i++)
        printf("%02d. %s\n", i + 1, *(cities + i));
}

```

```

int start, end;
do
{
    printf("\nВведите номер начального города: ");
    start = input_int_number() - 1;
    if (start < 0 || start >= city_count)
        printf("Неверный номер города!\n");
} while (start < 0 || start >= city_count);

do
{
    printf("Введите номер конечного города: ");
    end = input_int_number() - 1;
    if (end < 0 || end >= city_count)
        printf("Неверный номер города!\n");
    else if (end == start)
        printf("Начальный и конечный город не должны совпадать!\n");
} while (end < 0 || end >= city_count || end == start);

int* dist = (int*)malloc(city_count * sizeof(int));
int* time = (int*)malloc(city_count * sizeof(int));
int* prev = (int*)malloc(city_count * sizeof(int));

dijkstra_algorithm(distance_graph, time_graph, city_count, start, dist, time,
prev);

char* result = build_path_result(cities, dist, time, prev, city_count, end);

free_memory(cities, distance_graph, time_graph, dist, time, prev, city_count);

return result;
}
void dijkstra_algorithm(int** graph, int** time_graph, int city_count, int start, int*
dist, int* time, int* prev)
{
    int* visited = (int*)calloc(city_count, sizeof(int));

    for (int i = 0; i < city_count; i++)
    {
        *(dist + i) = INT_MAX;
        *(time + i) = INT_MAX;
        *(prev + i) = -1;
    }
    *(dist + start) = 0;

```

```

*(time + start) = 0;

for (int count = 0; count < city_count - 1; count++)
{
    int min_dist = INT_MAX;
    int u = -1;

    for (int v = 0; v < city_count; v++)
    {
        if (!*(visited + v) && *(dist + v) < min_dist)
        {
            min_dist = *(dist + v);
            u = v;
        }
    }

    if (u == -1) break;
    *(visited + u) = 1;

    for (int v = 0; v < city_count; v++)
    {
        if (!*(visited + v) && (*(graph + u) + v) != INT_MAX && *(dist + u) !=
INT_MAX)
        {
            int alt_dist = *(dist + u) + (*(graph + u) + v);
            int alt_time = *(time + u) + (*(time_graph + u) + v);

            if (alt_dist < *(dist + v))
            {
                *(dist + v) = alt_dist;
                *(time + v) = alt_time;
                *(prev + v) = u;
            }
        }
    }
}

free(visited);
}
char* build_path_result(char** cities, int* dist, int* time, int* prev, int city_count,
int end)
{
    if (*(dist + end) == INT_MAX)
    {

```

```

    char* result = (char*)malloc(50 * sizeof(char));
    strcpy_s(result, 50, "Путь между городами не найден.");
    return result;
}

int* path = (int*)malloc(city_count * sizeof(int));

int path_len = 0;
for (int at = end; at != -1; at = *(prev + at))
    *(path + path_len++) = at;

size_t total_len = 150; // Базовый размер
for (int i = path_len - 1; i >= 0; i--)
    total_len += strlen(*(cities + *(path + i))) + 4;
char* result = (char*)calloc(total_len, sizeof(char));

// Формирование маршрута
strcat_s(result, total_len, "Маршрут: ");
for (int i = path_len - 1; i >= 0; i--)
{
    strcat_s(result, total_len, *(cities + *(path + i)));
    if (i > 0)
        strcat_s(result, total_len, " -> ");
}

char info[100];
sprintf_s(info, 100, "\nРасстояние: %d км\nВремя в пути: %d мин. (%.1f ч.)\n", *(dist + end), *(time + end), *(time + end) / 60.0);
strcat_s(result, total_len, info);

free(path);
return result;
}

void free_memory(char** cities, int** distance_graph, int** time_graph, int* dist,
int* time, int* prev, int cities_count)
{
    for (int i = 0; i < cities_count; i++)
    {
        free(*(cities + i));
        free(*(distance_graph + i));
        free(*(time_graph + i));
    }
    free(cities);
}

```



```
    free(distance_graph);  
    free(time_graph);  
  
    free(dist);  
    free(time);  
    free(prev);  
}
```

ПРИЛОЖЕНИЕ 3
(обязательное)

Ведомость документов